附录核心源代码

正文后附：B* 树 / Skyline 解码 / 扰动 / 快速模拟退火 / 二分与双向确认 / 超块穷举

2026 年第七届” 华数杯” 大学生数学建模竞赛 B 题

2026/08/10

本附录列出与正文模型、算法对应的**核心函数源代码**（与提交源程序逐行一致），供代码复核与结果复现时对照。

1 节点数据结构（10-bit 位压缩）

```

template<typename ID, typename LEN> struct FLOOR_PLAN<ID, LEN>::NODE {
  NODE() : x(0) {};
  ID _l() const { return (x >> 01) & ((1 << 10) - 1); }
  ID _r() const { return (x >> 11) & ((1 << 10) - 1); }
  ID _p() const { return (x >> 21) & ((1 << 10) - 1); }
  bool _rot() const { return x & 1; }
  void s_l(const uint& i) { x = ((x & rmsk_l) | ((i << 01) & msk_l)); }
  void s_r(const uint& i) { x = ((x & rmsk_r) | ((i << 11) & msk_r)); }
  void s_p(const uint& i) { x = ((x & rmsk_p) | ((i << 21) & msk_p)); }
  void s_rot() { x ^= 1; }
  unsigned int x;
};
```

2 Skyline 轮廓压缩解码

```

  void dfs(ID id, list<ID>& cy, typename list<ID>::iterator& cur,
           vector<BLOCK>& blcks) {
    if (_nodes[id]._l()) {
      const ID& lc = _nodes[id]._l();
      blcks[lc]._x = blcks[id]._x + blcks[id]._w;
```

```
      blcks[lc]._y = find_max_y(cy, ++cur, blcks, blcks[lc]);
      dfs(_nodes[id]._l(), cy, cur, blcks);
      --cur;
    }
    if (_nodes[id]._r()) {
      const ID& rc = _nodes[id]._r();
      blcks[rc]._x = blcks[id]._x;
      blcks[rc]._y = find_max_y(cy, cur, blcks, blcks[rc]);
      dfs(_nodes[id]._r(), cy, cur, blcks);
    }
  }
  LEN find_max_y(list<ID>& l, typename list<ID>::iterator& cur,
                 vector<BLOCK>& blcks, BLOCK& blck) {
    LEN y = 0;
    auto it = cur, rit = cur;
    while (it != l.end() && blcks[*it]._x < blck._x + blck._w) {
      if (blcks[*it]._y + blcks[*it]._h > y) y = blcks[*it]._y + blcks[*
          it]._h;
      if (blcks[*rit]._x + blcks[*rit]._w <= blck._x + blck._w) ++rit;
      ++it;
    }
    cur = l.erase(cur, rit);
    cur = l.insert(cur, blck._id);
    return y;
  }
```

3 三种扰动算子

```

  void perturb() {
    float p1 = randf(), p2 = randf();
    if (p1 < _rot_prob) rotate();
    else if (p2 < _del_and_ins_prob) del_and_ins();
    else swap_two_nodes();
    _has_init = false;


  void rotate() {
    if (_rotable.empty()) return;
    ID id = _rotable[(rand() % _rotable.size())];
    _blcks[id]._rot = !_blcks[id]._rot;
    swap(_blcks[id]._w, _blcks[id]._h);
    _tree.rotate(id);
```

```
  }
  void del_and_ins() {
    if (_Nblcks < 2) return;
    ID id = (rand() % _Nblcks) + 1;
    _tree.del_from_tree(id);
    ID p = (rand() % _Nblcks) + 1;
    bool left = randb();
    while (id == p) p = (rand() % _Nblcks) + 1;
    _tree.ins_to_tree(p, id, left);
  }
  void swap_two_nodes() {
    if (_Nblcks < 2) return;
    ID id1 = rand() % _Nblcks;
    ID id2 = (id1 + ((rand() % (_Nblcks - 1)) + 1)) % _Nblcks;
    _tree.swap_two_nodes(id1 + 1, id2 + 1);
  }
```

4 代价函数与可行性判定

```

  float norm_cost(const int3& cost) const {
    const float r = float(get<2>(cost)) / get<1>(cost);
    const float area_term = _alpha * get<1>(cost) * get<2>(cost) /
        _avg_area;
    const float hpwl_term = (_avg_hpwl > 0) ? _beta * get<0>(cost) /
        _avg_hpwl / 2. : 0.f;
    const float r_term = (_avg_r > 0)
        ? (1 - _alpha - _beta) * (r - _R) * (r - _R) / _avg_r : 0.f;
    return area_term + hpwl_term + r_term;
  }
  float true_cost(const int3& cost, const float den = 1) const {
    return (_true_alpha * get<1>(cost) * get<2>(cost)
            + (1 - _true_alpha) * get<0>(cost) / 2.) / den;
  }
  bool feas(const int3& cost) {
    if (_mode == Mode::Q1) return true;
    return (get<1>(cost) <= _W && get<2>(cost) <= _H);
  }
```

5 快速模拟退火（精修阶段 run2 主循环）

```
1
2   run2(const int k, int rnd, const float c) {
3     float _init_T2 = _init_T / ((_mode == Mode::Q1) ? 20.f : 50.f);
4     int reset_th = 2 * _Nblcks, stop_th = 9 * _Nblcks, reset_cnt = 0;
5     int iter = 1, tot_feas = 0, rej_num = 0, cnt = 1;
6     _fp.init();
7     tot_feas = feas(_fp.cost()) ? 1 : 0;
8     float _T = _init_T2;
9     float prv_cost = (_mode == Mode::Q1) ? q1_cost(_fp.cost()) :
          true_cost(_fp.cost(), _avg_true);
10    _best_cost = prv_cost;
11  #ifdef TREE_DEBUG
12    _dbg_hist.push_back(-1.0f);
13  #endif
14    dbg_push(_best_cost);
15    typename FLOOR_PLAN<ID, LEN>::TREE last_sol = _best_sol;
16    while (_T > temp_th || float(rej_num) <= rej_ratio * cnt || !tot_feas
          ) {
17      float avg_delta_cost = 0;
18      rej_num = 0, cnt = 1;
19      for (; cnt <= rnd; ++cnt) {
20        _fp.perturb();
21        _fp.init();
22        int3 costs = _fp.cost();
23        float cost = (_mode == Mode::Q1) ? q1_cost(costs) : true_cost(
              costs, _avg_true);
24        float delta_cost = (cost - prv_cost);
25        avg_delta_cost += abs(delta_cost);
26
27        if (delta_cost <= 0 || randf() < expf(-delta_cost / _T)) {
28          prv_cost = cost;
29          last_sol = _fp.get_tree();
30          if (feas(costs)) {
31            ++tot_feas;
32            if (cost < _best_cost) {
33              _best_sol = _fp.get_tree();
34              _best_cost = cost;
35              dbg_push(_best_cost);
36            }
37          }
38        } else {
39          _fp.restore(last_sol);
40          ++rej_num;
41        }
```

```
      }
      ++iter;
      if (iter <= k) _T = _init_T2 * avg_delta_cost / cnt / iter / c;
      else _T = _init_T2 * avg_delta_cost / cnt / iter;
      _fp.init();
      dbg_log("run2", iter, _T, _alpha, tot_feas ? 1 : 0);
      if (_log) {
        _snap_iters.push_back(iter);
        _snap_trees.push_back(_best_sol);
      }
      if (reset_cnt > _Nblcks / 7 + 1) break;
      if (!tot_feas) {
        if (iter > reset_th) {
          _T = _init_T2;
          iter = 1;
          ++reset_th, ++stop_th, ++rnd, ++reset_cnt;
        }
      } else if (iter > stop_th) break;
    }
    _fp.restore(_best_sol);
    _fp.init();
    dbg_snapshots();
    int3 costs = _fp.cost();
    _best_cost = (_mode == Mode::Q1) ? q1_cost(costs) : true_cost(costs);
    if (_mode == Mode::Q1) dbg_push(q1_cost(costs));
    else dbg_push(true_cost(costs, _avg_true));
    return {_best_cost, _best_sol};
  }
```

6 求解入口

```

          ostream* log = nullptr, bool feas_only = false) {
  FLOOR_PLAN<ID, LEN> fp(fnets, fblcks, fpl, rpt, Nnets, Nblcks, Ntrmns,
                         alpha, dead_ratio, mode == Mode::Q1);
  float P = 0.9f, alpha_base = 0.5f, beta = 0.1f;
  const float R = fp.R();
  int k = max(2, Nblcks / 11), rnd = 2 * Nblcks + 20;
  float c = max(100 - int(Nblcks), 10);
  SA<ID, LEN> sa(fp, Nblcks, fp.W(), fp.H(), R, P, alpha_base, beta,
      alpha,
                 mode, log);
  if (feas_only) {
```

```
    sa.run(k, rnd, c, true);
    fp.init();
    int3 costs = fp.cost();
    ofstream outs(rpt);
    outs << (sa.last_feasible() ? 1 : 0) << '\n';
    outs << get<1>(costs) << " " << get<2>(costs) << '\n';
    outs << get<0>(costs) / 2. << '\n';
    return;
  }
  typename FLOOR_PLAN<ID, LEN>::TREE trees[2];
  float costs[2];
  if (mode == Mode::Q1) {
    for (int i = 0; i < 2; ++i) {
      tie(costs[i], trees[i]) = sa.run2(k, rnd, c);
    }
  } else {
    for (int i = 0; i < 2; ++i) {
      sa.run(k, rnd, c);
      tie(costs[i], trees[i]) = sa.run2(k, rnd, c);
    }
  }
  fp.restore(costs[0] < costs[1] ? trees[0] : trees[1]);
  fp.init();
  ofstream outs(rpt);
  fp.output(outs);
```

7 问题三：二分搜索与双向确认

```

def bisect(chip, total, work_dir, trace_path, eps=EPS,
           coarse_seeds=COARSE_SEEDS, precise_seeds=PRECISE_SEEDS):
    """二分 [D_LO, D_HI], <eps 早停。返回 (d_star, 迭代步数)。"""
    lo, hi = D_LO, D_HI
    rows = []
    it = 0
    while hi - lo >= eps:
        it += 1
        mid = (lo + hi) / 2.0
        n_seeds = coarse_seeds if (hi - lo) >= PRECISE_THRESHOLD else
            precise_seeds
        t0 = time.time()
        feasible, results = feas_check_parallel(chip, mid, total,
            work_dir,
```

```
                                                n_seeds, it)
        dt = time.time() - t0
        bb = results[0][1]
        if feasible:
            hi = mid
        else:
            lo = mid
        rows.append([it, lo, hi, mid, side_for(mid, total),
                     f"{bb[0]}x{bb[1]}", int(feasible), n_seeds, round(dt
                         , 1)])
    with open(trace_path, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["iter", "lo", "hi", "mid", "side", "bbox", "feasible"
            ,
                    "attempts", "time_s"])
        w.writerows(rows)
    return hi, it


def confirm_minimum(chip, d_star, total, work_dir, verify_seeds=
    VERIFY_SEEDS):
    """双向确认: d* 本身必须可行（不可行则上移 +EPS 重验）；
    d*-EPS 处多种子必须全不可行（有可行则下移重验）。
    返回 (final_d, 确认记录)。"""
    steps = []
    for k in range(12):
        feas_here, r_here = verify_at(chip, d_star, total, work_dir, 900
            + k,
                                      n_seeds=verify_seeds)
        if not feas_here:
            d_star = min(D_HI, d_star + EPS)
            steps.append({"d": round(d_star, 6), "d_feas": False,
                          "note": "up-shift"})
            continue
        below = max(0.0, d_star - EPS)
        feas_below, r_below = verify_at(chip, below, total, work_dir, 950
            + k,
                                        n_seeds=verify_seeds)
        steps.append({"d": round(d_star, 6), "below": round(below, 6),
                      "d_feas": True, "below_infeas": not feas_below,
                      "below_results": [r[0] for r in r_below]})
        if not feas_below:
            return d_star, steps
        d_star = below
    return d_star, steps
```

8 问题四：超块穷举（矩形分解与凹角支撑解码）

```python

def rotate_90(rects):
    """模块整体逆时针旋转 90 度（角点变换 + 归一化平移）。"""
    new = []
    for (x, y, w, h) in rects:
        pts = [(x, y), (x + w, y), (x, y + h), (x + w, y + h)]
        nps = [(-yy, xx) for (xx, yy) in pts]
        minx = min(p[0] for p in nps)
        maxx = max(p[0] for p in nps)
        miny = min(p[1] for p in nps)
        maxy = max(p[1] for p in nps)
        new.append((minx, miny, maxx - minx, maxy - miny))
    minx = min(r[0] for r in new)
    miny = min(r[1] for r in new)
    return [(x - minx, y - miny, w, h) for (x, y, w, h) in new]


def place(shape, perm, rots):
    """按树形+排列+旋转放置 4 超块，返回 (合法?, 包围盒面积, 每模块 [(name, rot, x, y, rects)])."""
    root, P, L, R = shape
    placed = []           # 全局已放置矩形 (x, y, w, h, owner)
    mods = []             # 每模块: (name, rot_idx, x, y, rects_abs)
    xs = [0] * 4
    ys = [0] * 4

    def support(rect):
        """rect 需要的 y 支撑顶: 与已放置矩形 x 区间相交的最大顶。"""
        top = 0
        rx0, ry0, rw, _ = rect
        for (px, py, pw, ph, _) in placed:
            if px < rx0 + rw and rx0 < px + pw:
                if py + ph > top:
                    top = py + ph
        return top

    def dfs(sid):
        name = NAMES[perm[sid]]
        rects = ROTATIONS[name][rots[sid]]
        if sid == root:
```

```
            xs[sid] = 0
        else:
            p = P[sid]
            pname = NAMES[perm[p]]
            pw, _ = bbox_of(ROTATIONS[pname][rots[p]])
            if L[p] == sid:
                xs[sid] = xs[p] + pw
            else:
                xs[sid] = xs[p]
        # 超块 y = max(内部矩形所需 y)
        y = 0
        for (rx, ry, rw, rh) in rects:
            need = support((xs[sid] + rx, ry, rw, rh)) - ry
            if need > y:
                y = need
        ys[sid] = y
        for (rx, ry, rw, rh) in rects:
            placed.append((xs[sid] + rx, y + ry, rw, rh, name))
        mods.append((name, rots[sid], xs[sid], y, rects))
        if L[sid] != -1:
            dfs(L[sid])
        if R[sid] != -1:
            dfs(R[sid])

    dfs(root)
    # 矩形级重叠检查（6 实际矩形两两）
    for i in range(len(placed)):
        for j in range(i + 1, len(placed)):
            a, b = placed[i], placed[j]
            if a[0] < b[0] + b[2] and b[0] < a[0] + a[2] and \
               a[1] < b[1] + b[3] and b[1] < a[1] + a[3]:
                return False, 0, None
    maxx = max(r[0] + r[2] for r in placed)
    maxy = max(r[1] + r[3] for r in placed)
    return True, maxx * maxy, mods
```